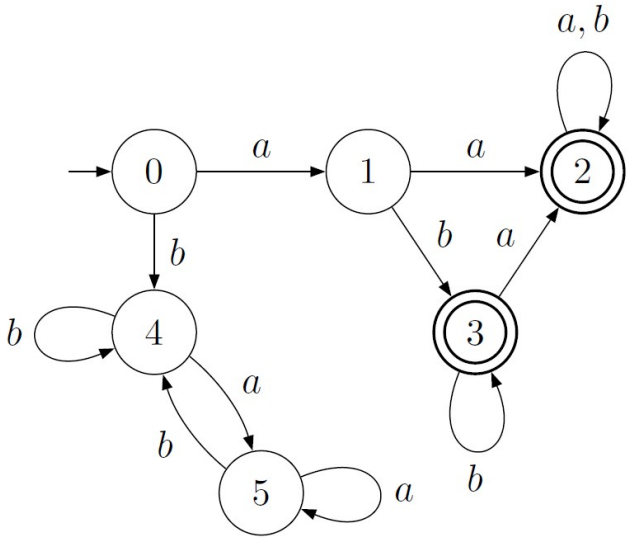


G52LAC (COMP2040)
Languages and Computation

Lecture 10:
Turing Machines, Decidability & Computability Theory

Tomas Maul
School of Computer Science
University of Nottingham Malaysia Campus

Outline

- Wrap-up previous lecture.
- Context-sensitive grammars.
- Turing Machines.
- Decidability.
- Lambda calculus.
- P versus NP

Outline

- Wrap-up previous lecture.
- Context-sensitive grammars.
- Turing Machines.
- Decidability.
- Lambda calculus.
- P versus NP

Recap: CFGs

A Context-Free Grammar is a 4-tuple (N, Σ, P, S) where:

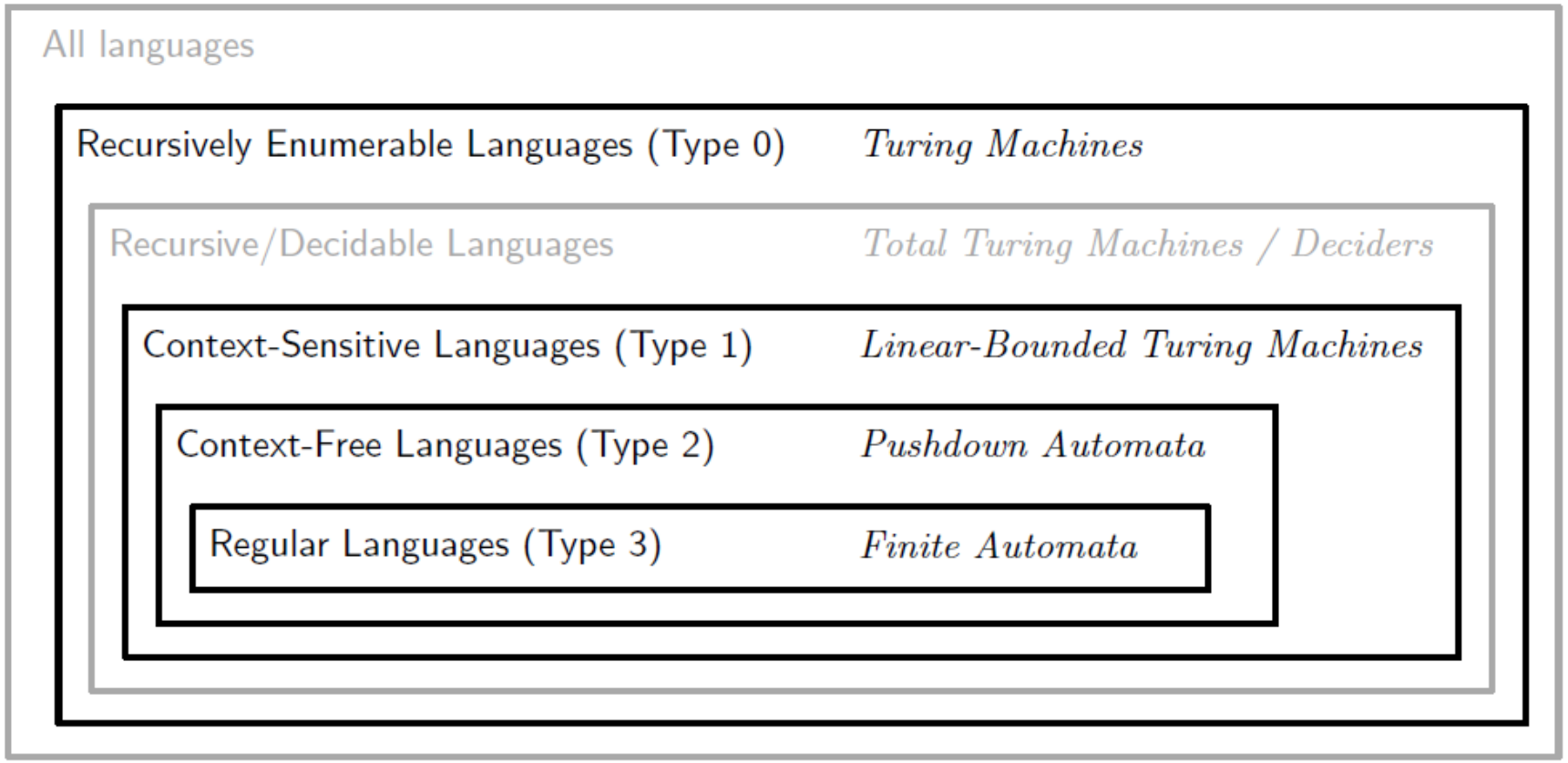
- N is a finite set of non-terminal symbols
- Σ is a finite set of terminal symbols
- $N \cap \Sigma = \emptyset$
- P is a finite set of productions. Each production has the form $A \rightarrow \beta$, where $(A \in N), (\beta \in (N \cup \Sigma)^*)$
- $S \in N$ is the start symbol

Limitations of Context-Free Languages

- The Context-Free Languages are those that can be recognised by Pushdown Automata.
- Unlike finite automata, they allow an unlimited amount of pairs of symbols to be matched up. e.g. 0^n1^n
- However, there are some simple languages that they cannot express, e.g.

$$\{a^n b^n c^n \mid n \in \mathbb{N}\}$$

The Chomsky Hierarchy



Context-Sensitive Grammars (CSGs)

Essentially, a Context-Sensitive Grammar is like a CFG, except that the productions:

- may have multiple symbols on the left-hand side;
- must be non-contracting (at least as many symbols on the right as the left).

Formal Definition of a CSG

A Context-Sensitive Grammar is a 4-tuple (N, Σ, P, S) where:

- N is a finite set of non-terminal symbols.
- Σ is a finite set of terminal symbols.
- $N \cap \Sigma = \emptyset$
- P is a finite set of productions.
  Each production has the form $\alpha \rightarrow \beta$ such that:
  - $\alpha, \beta \in (N \cup \Sigma)^+$
  - $|\alpha| \leqslant |\beta|$

  Also, there may be an additional production $S \rightarrow \varepsilon$, provided S does not appear on the right-hand side of any production.
- $S \in N$ is the start symbol.

Otherwise, the non-contracting rule can be violated

The Language of a CSG

- Given a CSG $G = (N, \Sigma, P, S)$, the language generated by G is defined as:

$$L(G) = \{w \mid (w \in \Sigma^*) \wedge (S \overset{+}{\underset{G}{\Rightarrow}} w)\}$$

- A language C is a Context-Sensitive Language iff there exists a CSG G such that $C = L(G)$

Example CSG

$L = \{ a^n b^n c^n : n >= 0 \}$

S → aBCT | aBC
T → ABCT | ABC
BA → AB
CA → AC
CB → BC
aA → aa
aB → ab
bB → bb
bC → bc
cC → cc

Try:

abc

aabbcc

Example CSG

$L = \{ a^n b^n c^n : n >= 0 \}$

S → aBCT | aBC
T → ABCT | ABC
BA → AB
CA → AC
CB → BC
aA → aa
aB → ab
bB → bb
bC → bc
cC → cc

aabbcc

General operation: (1) get the right number of balanced “a”, “b” and “c”; (2) shuffle non-terminals to the left/right until they are in the correct position; (3) convert non-terminals (e.g. A) into lower case (e.g. a).

S => aBCT => aBCABC => aBACBC => aABCBC
=> aABBCC => aaBBCC => aabBCC => aabbCC
=> aabbcC => aabbcc

Important note. In this case (unlike what we did with CFGs) we are not imposing any restrictions in terms of leftmost/rightmost derivations.

Unrestricted Grammars

- More general than CSGs are Unrestricted Grammars.
- Unrestricted grammars express the Recursively Enumerable languages.
- Key difference: no restriction on the length of the right-hand side of the productions.

Formal Definition of an Unrestricted Grammar

An Unrestricted Grammar is a 4-tuple (N, Σ, P, S) where:

- N is a finite set of non-terminal symbols.
- Σ is a finite set of terminal symbols.
- $N \cap \Sigma = \emptyset$
- P is a finite set of productions.
  Each production has the form $\alpha \rightarrow \beta$ where:
  - $\alpha \in (N \cup \Sigma)^+$
  - $\beta \in (N \cup \Sigma)^*$
- $S \in N$ is the start symbol.

Recommended Reading

- G52MAL Lecture Notes, pages 53–54

Outline

- Wrap-up previous lecture.
- Context-sensitive grammars.
- Turing Machines.
- Decidability.
- Lambda calculus.
- P versus NP

Turing Machines (TMs)

Turing Machines:

- A mathematical model of a general-purpose computer (with infinite memory)
- Mainly used to study the notion of computation (what computers can and can't do)

The Church-Turing Thesis:

Every function which would naturally be regarded as 'computable' can be computed by a Turing Machine.

Informal Definition

- A Turing Machine is a finite automaton with access to an infinite tape:

$$TM \approx Finite\ Automaton + Infinite\ Tape$$

- Initially the tape contains the input (word), but is otherwise blank.
- The tape has a pointer marking the current position, which starts at the first input symbol.
- During operation, the pointer moves left and right, reading and writing symbols on the tape.
- A TM accepts as soon as it enters an accepting state.

Formal Definition

A Turing Machine is a 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where:

- Q is a finite set of states
- Σ is the input alphabet
- Γ is the tape alphabet
- $\delta \in Q \times \Gamma \rightarrow (Q \times \Gamma \times \{\mathbf{L}, \mathbf{R}\}) \cup \{\mathbf{stop}\}$ is a transition function
- $q_0 \in Q$ is the initial state
- $B \in \Gamma$ is the blank symbol
- $F \subseteq Q$ is the set of accepting states
- Additionally, $B \notin \Sigma$ and $\Sigma \subset \Gamma$

Instantaneous Descriptions (IDs)

- Like PDAs, we can give an ID completely describing the state of computation of a TM.
- Definition: an Instantaneous Description is a triple

$$(\alpha, q, \beta) \in \Gamma^* \times Q \times \Gamma^*$$

where

  - q is the current state
  - α is the non-blank part of the tape to the left of the current position
  - β is the non-blank part of the tape to the right of, and including, the current position

Next State Relation

As for PDAs we define instantaneous descriptions ID for Turing machines. We have $\mathrm{ID} = \Gamma^* \times Q \times \Gamma^*$ where $(\gamma_L, q, \gamma_R) \in \mathrm{ID}$ means that the TM is in state q and left from the head the non-blank part of the tape is γ_L and starting with the head itself and all the non-blank symbols to the right is γ_R.

We define the next state relation $\underset{M}{\vdash}$ similar as for PDAs:

1. $(\gamma_L, q, x\gamma_R) \underset{M}{\vdash} (\gamma_L y, q', \gamma_R)$ if $\delta(q, x) = (q', y, R)$
2. $(\gamma_L z, q, x\gamma_R) \underset{M}{\vdash} (\gamma_L, q', zy\gamma_R)$ if $\delta(q, x) = (q', y, L)$
3. $(\epsilon, q, x\gamma_R) \underset{M}{\vdash} (\epsilon, q', By\gamma_R)$ if $\delta(q, x) = (q', y, L)$
4. $(\gamma_L, q, \epsilon) \underset{M}{\vdash} (\gamma_L y, q', \epsilon)$ if $\delta(q, B) = (q', y, R)$
5. $(\gamma_L z, q, \epsilon) \underset{M}{\vdash} (\gamma_L, q', zy)$ if $\delta(q, B) = (q', y, L)$
6. $(\epsilon, q, \epsilon) \underset{M}{\vdash} (\epsilon, q', By)$ if $\delta(q, B) = (q', y, L)$

The current symbol x is replaced by y.

We are here talking about deterministic Turing Machines.

The cases 3 to 6 are only needed to deal with the situation of having reached the end of the non-blank part of the tape.

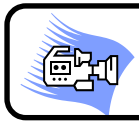
The Language of a Turing Machine

Given a TM $T = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, the language of T is defined as:

$$L(T) = \{w \mid (w \in \Sigma^*) \wedge ((\varepsilon, q_0, w) \overset{*}{\underset{T}{\vdash}} (\alpha, q, \beta)) \wedge (q \in F)\}$$

Consequently, a TM:

- stops and accepts the word as soon as it enters an accepting state;
- is not guaranteed to stop if the word is not in the language.



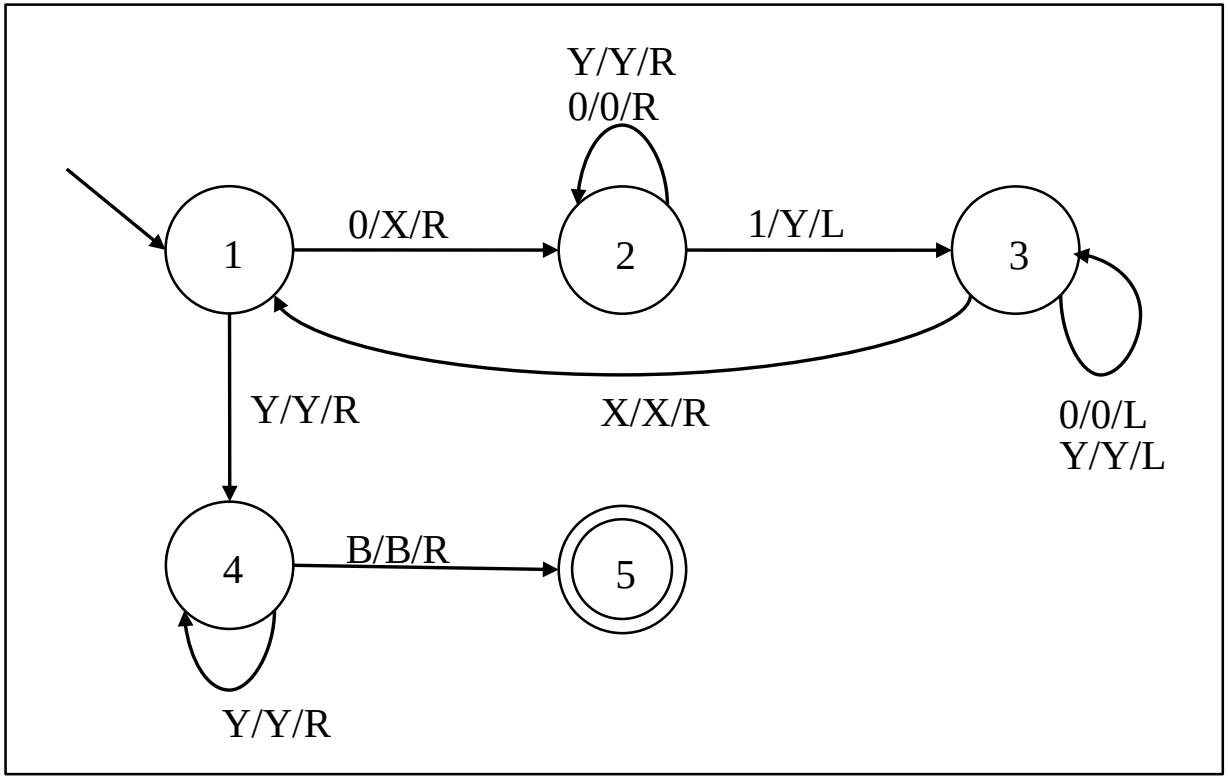
http://www.youtube.com/watch?v=E3keLeMwfHY

Example Turing Machine

0/X/R: see 0, write X, move scanner head to the right

Example.
Language: any ideas?

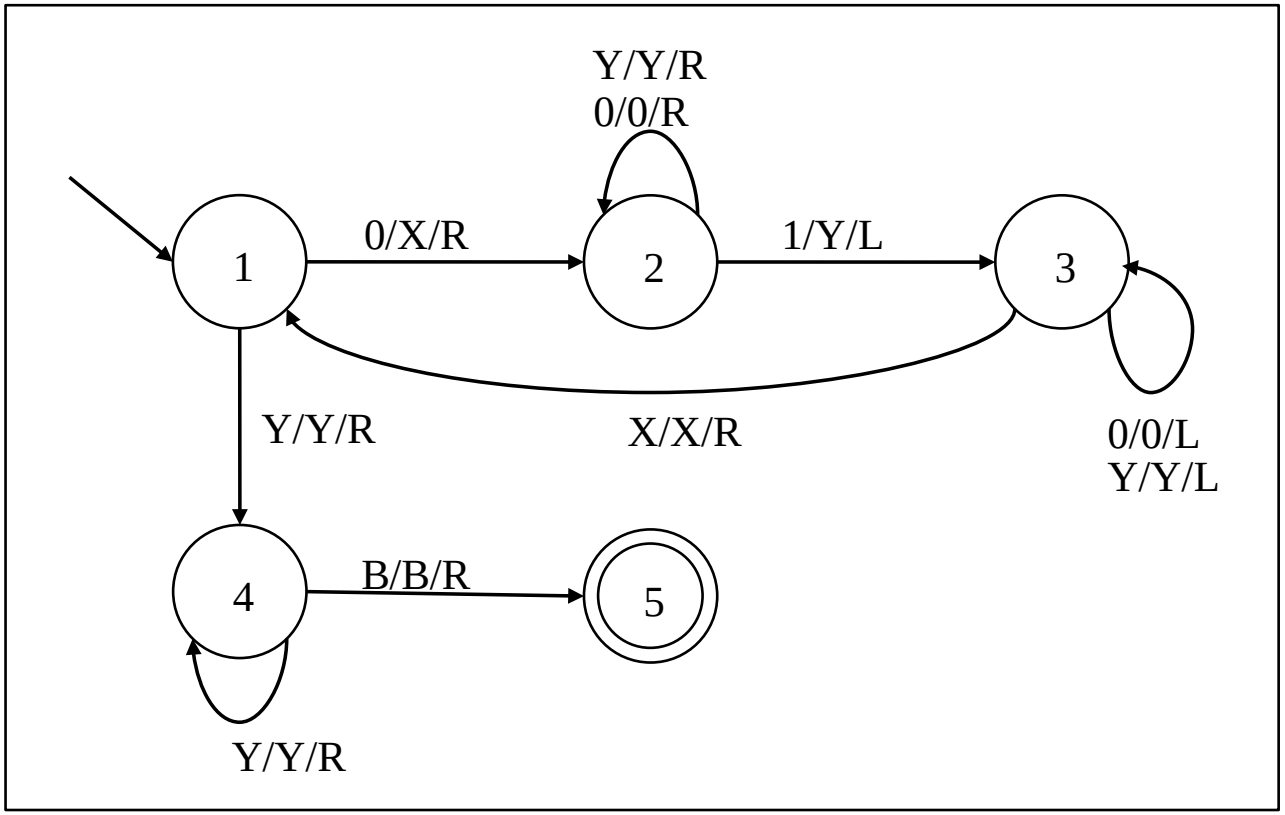
Explain basic operation.



Example Turing Machine

0/X/R: see 0, write X, move scanner head to the right

Language: 0^n1^n



Try: 0011

Try also:
00011 and
00111

Deterministic and Non-Deterministic TMs

- The Turing Machine model described in this lecture is deterministic.
- However, like finite automata (and unlike PDAs), deterministic TMs have the same expressive power as non-deterministic TMs.

From the module's pdf notes: "Context-sensitive languages (...) correspond to linear bounded TMs, these are those TMs that use only a tape whose length can be given by a linear function over the length of the input."

Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 324–335
- G52MAL Lecture Notes, pages 49–53

Outline

- Wrap-up previous lecture.
- Context-sensitive grammars.
- Turing Machines.
- Decidability.
- Lambda calculus.
- P versus NP

Language Membership

- Problems can be expressed as membership in a language.
- E.g. to determine if an integer is prime, ask if it is in the language of prime numbers.
- Yet many problems require output other than yes/no.
- For Turing Machines, the contents of the tape after halting can be taken as the output.
- However, language membership is still a useful measure of complexity.
  - E.g. a parser builds an Abstract Syntax Tree for a program.
  - Doing so must be at least as complex as checking it is in the language.

Halting

From Hopcroft et al, page 334: "(...) If the TM eventually enters an accepting state, then the input is accepted, and otherwise not. (...) We say a TM halts if it enters a state q, scanning a tape symbol X, and there is no move in this situation; i.e. $\delta(q,X)$ is undefined. (...) We assume that a TM halts when it is in an accepting state. Unfortunately, it is not always possible to require that a TM halts even if it does not accept. (...)"

Again: For a TM to halt but not accept a word it needs to enter a state, which is not final, and have no move in this situation.

- Some TMs halt for all inputs.
- Some TMs fail to halt for some inputs.
- Some languages are such that any TM recognising the language cannot halt for some inputs.

Based on these observations, it makes to define several classes of Turing Machines and Languages.

Recursively Enumerable Languages

- A language R is recursively enumerable if there exists a TM T such that $R = L(T)$
- Recall that for any Turing Machine T:
  - if $w \in L(T)$ then T is guaranteed to halt and accept w
  - if $w \notin L(T)$ then T will either halt and reject w, or never halt
- Why "recursively enumerable"?
  - Because you can construct a TM that enumerates all words in the language.
  - But it may never halt, so you don't know if it's finished.
- Languages that have no corresponding Turing Machine are called non-recursively enumerable languages.

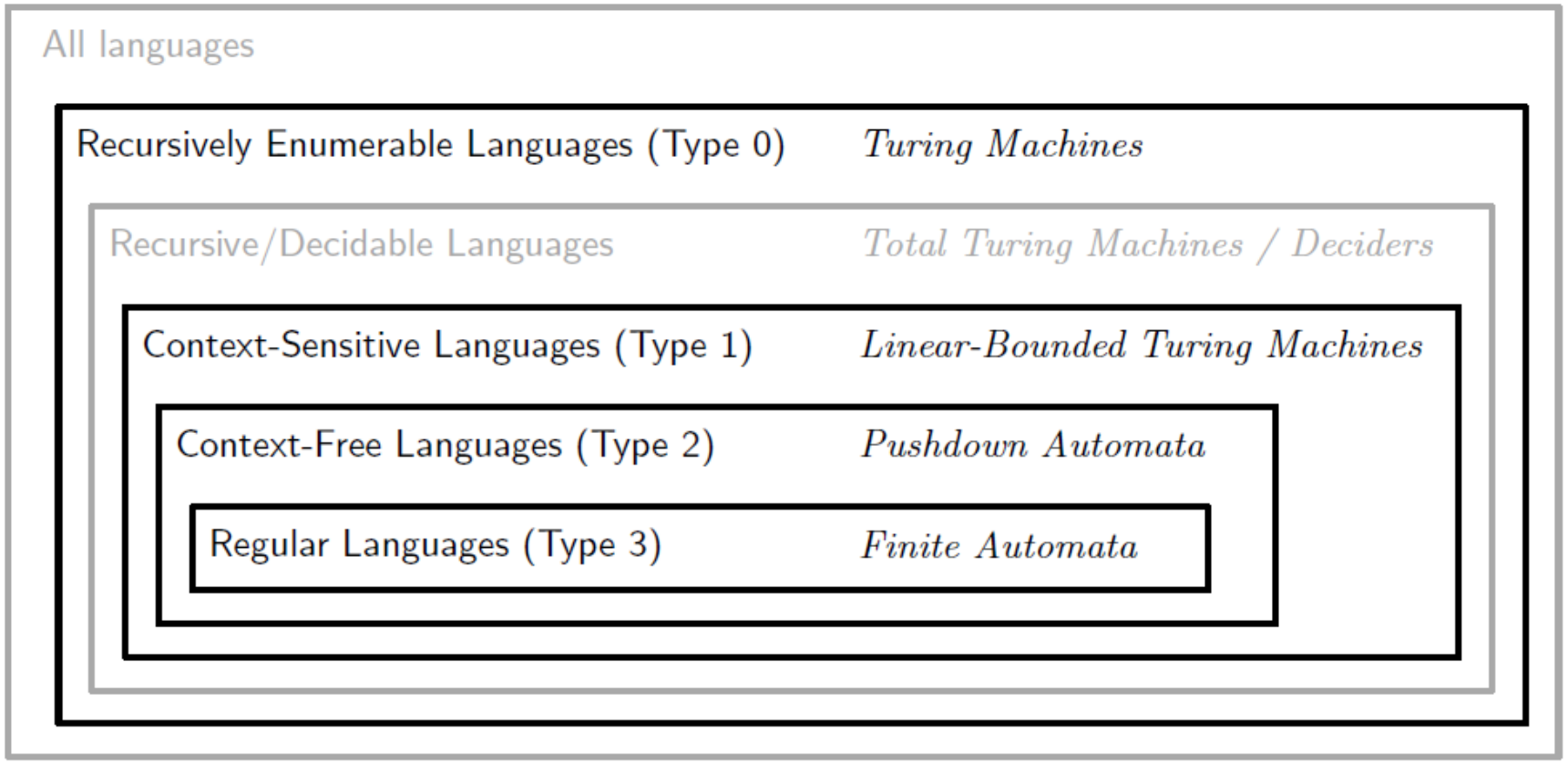
Total Turing Machines (Deciders)

- A Total Turing Machine is a TM that always halts (accepting if the word is in the language, rejecting if not).
- Such a TM is said to decide its language.

Recursive (Decidable) Languages

- A language R is recursive if there exists a Total Turing Machine T such that $R = L(T)$
- Why "recursive"?
  - It's the class of languages (or problems) that can be solved by terminating recursive functions.
- A language (or problem) that is recursive is said to be decidable.
- A language (or problem) that is not recursive is said to be undecidable.

The Chomsky Hierarchy



The Halting Problem

- The Halting Problem is a famous example of a recursively enumerable language that is not recursive.
- As a language recognition problem:

  Is there a TM that decides the language of terminating TMs?

- Or to put it another way:

  Can we write a program that takes the text of another program as its input, along with an input to that second program, and decides whether that second program terminates for the given input?

- The answer is NO!

Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 315–324 & 377–390
- G52MAL Lecture Notes, pages 54–56

Outline

- Wrap-up previous lecture.
- Context-sensitive grammars.
- Turing Machines.
- Decidability.
- Lambda calculus.
- P versus NP.

Lambda calculus

Very brief introduction

- Pure notion of effective computation procedure: **all** computation reduced to function definition and application.
- Can be seen as the smallest universal programming language in the world (Rojas, 1997).
- Invented in the 1930s by Alonzo Church.
- Comparable to other formalisations of the notion of effective computation; e.g., the Turing machine.
- The λ-calculus and Turing Machines are equivalent in that they capture the exact same notion of what “computation” means.

Lambda calculus

Very brief introduction

• The Church-Turing Hypothesis: The λ-calculus, Turing Machines, etc. coincides with our intuitive understanding of what “computation” means.
• The λ-calculus is important because it is at once:

- very simple, yet in essence a practically useful programming language
- mathematically precise, allowing for formal reasoning.

Lambda calculus

Very brief introduction

- The λ-calculus consists of:
  - one simple transformation rule (variable substitution) and
  - one simple function definition scheme.
- Only two keywords: lambda (λ) and dot (.).
- Key concepts: expressions and names (aka variables).

Expression defined recursively as:

<expression> := <name> | <function> | <application>
<function> := λ <name>.<expression>
<application> := <expression><expression>

- Brackets can be used for clarity. If E is an expression, then (E) is the same expression.
- Function application associates from the left.
  - $E_1E_2E_3 \ldots E_n$ is evaluated as $(\ldots ((E_1E_2)E_3) \ldots E_n)$

Lambda calculus

Very brief introduction

The syntax in the previous page can be expressed in the following CFG:

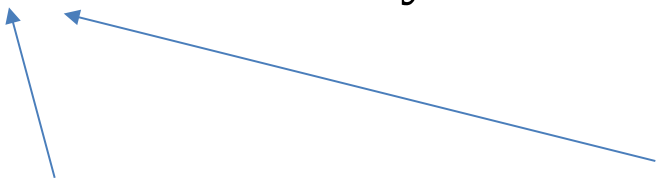
exp → ID
exp → (exp)
exp → λ ID . exp // function
exp → exp exp // application

Lambda calculus

Very brief introduction

<expression> := <name> | <function> | <application>
<function> := λ <name>.<expression>
<application> := <expression><expression>

- Example function:
  - λx.x ☐ identity function.



argument (after the λ)

body of the definition (after the dot)

- Functions can be applied to expressions, e.g.:
  - (λx.x)y ☐ application of the identity function to y.

Completing the application:

(λx.x) y = y

Lambda calculus

Very brief introduction

<expression> := <name> | <function> | <application>
<function> := λ <name>.<expression>
<application> := <expression><expression>

Another example:

λx.x+1 - function for adding 1 to the argument

Note that this is not a pure lambda calculus function because of the + operator.

Applying to the argument 3:

(λx.x+1)3 = 3+1 = 4

Lambda calculus

Very brief introduction

- Sources:
  - Henrik Nilsson lecture slides.
  - A Tutorial Introduction to the Lambda Calculus, by Raul Rojas, FU Berlin, WS-97/98.
    - http://www.inf.fu-berlin.de/inst/ag-ki/rojas_home/documents/tutorials/lambda.pdf

Outline

- Wrap-up previous lecture.
- Context-sensitive grammars.
- Turing Machines.
- Decidability.
- Lambda calculus.
- P versus NP.

P vs. NP

Very brief introduction

- P = NP ?
- Whether P equals NP is one of the most important outstanding open questions in Computer Science.
- One of the 7 Millenium Prize Problems.

The Clay Mathematics Institute will award 1 million dollars to those able to solve any of the problems (in this case either prove or disprove the equality P=NP).

P vs. NP

- P = set of problems with algorithms for finding solutions efficiently (at most in polynomial time).
- Example of a problem in P:

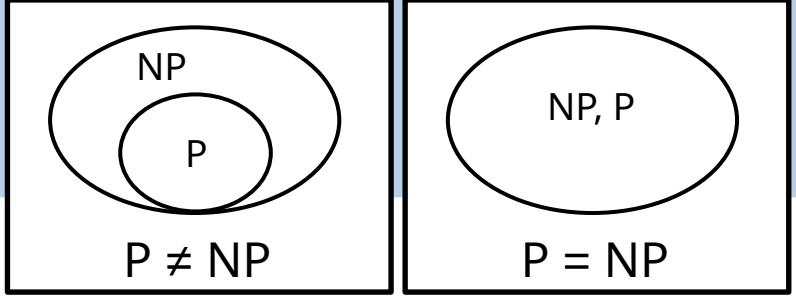
Input: a natural number x,
Question: is x in Even = {0,2,4,6,⋯}?

(Wikipedia) "An algorithm is said to be of **polynomial time** if its running time is upper bounded by a polynomial expression in the size of the input for the algorithm, i.e., $T(n) = O(n^k)$ for some constant k."

(Wikipedia) "An algorithm is said to be **exponential time**, if $T(n)$ is upper bounded by $2^{\text{poly}(n)}$, where poly(n) is some polynomial in n."

Source: http://cs.stackexchange.com/questions/9556/what-is-the-definition-of-p-np-np-complete-and-np-hard

P vs. NP



Very brief introduction

- NP = Problems with Efficient Algorithms for **Verifying** Proofs/Certificates/Witnesses (NP: non-deterministic polynomial time).
- Sometimes we don't know of an efficient algorithm that can solve our problem (all known algorithms take exponential time), but if we are given an answer/proof, we can verify it efficiently (in polynomial time). Problems of this nature are in the set NP.

No known efficient algorithm for answering the decision question for arbitrary sets in S, but efficient to verify a given solution, e.g. S={3,4,5,6}, A={3,6}, B={5,4}.

Partition
Input: a finite set of natural numbers S.
Question: is it possible to partition S into two sets A and B (where A∪B=S and A∩B=∅) such that the sum of the numbers in A is equal to the sum of numbers in B?

Source: http://cs.stackexchange.com/questions/9556/what-is-the-definition-of-p-np-np-complete-and-np-hard

- Note that P is a subset of NP. All problems in P are also in NP. The fundamental question is, are all problems in NP (efficient verification) also in P (have efficient algorithms for solving), thus implying P=NP?
  - Will all of these problems with easily verifiable solutions, eventually all be found to have efficient algorithms for computing solutions (P=NP) or not (P ≠NP)?
  - Most people believe in P ≠NP, but the proof is still waiting.
- Some experts would have preferred to name NP as VP (verifiable in polynomial time).
- Example NP problem:

P vs. NP

Very brief introduction

- Note that this is not the full story (e.g. NP-complete and NP-hard). This is just a very brief introduction.
- Main Source:
  - introduction to P, NP, NP-complete and NP-hard by Kaveh Ghasemloo, in a cs.stackexchange.com answer.

Revision

Any particular topics?

Acknowledgements

- Thorsten Altenkirch.
- Neil Sculthorpe.
- Henrik Nilsson.
- V.P. Kallimani.
- Venanzio Capretta.